

软件测试知识汇总

□ 编者注：本文档整合了软件测试五门核心课件内容，全部翻译为中文并添加注释，便于学习和复习。

第一章 软件测试基础知识

1.1 软件测试的定义

软件测试是通过手工或自动化手段，对软件系统进行验证（Verification）和确认（Validation）的过程。

□ 注释：验证是检查软件是否按照规格说明正确构建（“做得正确吗”），确认是检查软件是否满足用户真实需求（“做了正确的事吗”）。

1.2 软件测试的七条基本原则

编号	原则	含义
1	测试证明缺陷存在，而非不存在	即使测试通过，也不能证明软件没有 Bug
2	穷举测试是不可能的	除了极简单的程序，不可能测完所有输入组合
3	尽早测试	缺陷发现越早，修复成本越低
4	缺陷集群效应	大部分缺陷集中在少数模块中（二八原则）
5	杀虫剂悖论	反复使用同一测试用例会发现越来越少的新缺陷
6	测试依赖上下文	电商系统和航空控制系统测试策略完全不同
7	“零缺陷”是谬误	更重要的是软件是否满足用户需求

□ 注释：这七条原则是 ISTQB（国际软件测试资质认证委员会）的核心教条，考试和面试的高频考点。

1.3 软件测试的分类

软件测试

├── 按阶段划分

- ├── 单元测试 (Unit Testing)
- ├── 集成测试 (Integration Testing)
- ├── 系统测试 (System Testing)
- └── 验收测试 (Acceptance Testing)

├── 按方法划分

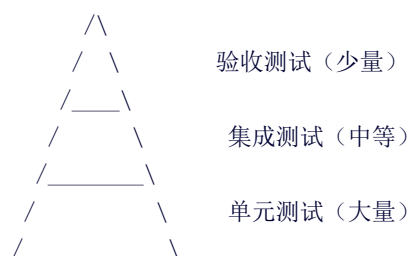
- ├── 黑盒测试 (Black-box Testing)
- ├── 白盒测试 (White-box Testing)
- └── 灰盒测试 (Gray-box Testing)

└── 按目的划分

- ├── 功能测试 (Functional Testing)
- └── 非功能测试 (Non-functional Testing)
 - ├── 性能测试
 - ├── 安全性测试
 - ├── 可用性测试
 - └── 兼容性测试

□ 注释：实际项目中通常会混合使用多种测试方法。单元测试和白盒测试通常由开发人员编写，系统测试和验收测试由测试团队负责。

1.4 测试金字塔



□ 注释：测试金字塔表明：单元测试应该最多（运行快、成本低），UI/验收测试最少（运行慢、维护成本高）。倒金字塔是反模式。

第二章 测试技术总览

2.1 黑盒测试技术 (Black-box Testing)

黑盒测试不关心内部结构，只关注输入 → 输出是否符合预期。

(1) 等价类划分 (Equivalence Partitioning)

将输入域划分为若干等价类，从每个类中选取代表值进行测试。

示例：输入成绩 0-100（整数）

有效等价类：0-100

无效等价类1：< 0（如 -5）

无效等价类2：> 100（如 105）

无效等价类3：非整数（如 "abc"）



注释：等价类划分的核心思想是——同一类中的值在程序中的行为应相同，所以只需测试代表值。

(2) 边界值分析 (Boundary Value Analysis)

缺陷往往出现在边界附近，应重点测试边界值。

示例（成绩 0-100）：

边界值：-1, 0, 1, 99, 100, 101

一般取 最小值-1、最小值、最大值、最大值+1

☐ 注释：等价类划分和边界值分析通常配合使用，先划分等价类，再找边界。

(3) 决策表测试 (Decision Table Testing)

将复杂条件组合以表格形式表达，确保每种组合都被测试。

条件	规则1	规则2	规则3	规则4
会员?	Y	Y	N	N
金额≥100?	Y	N	Y	N
动作	打9折+包邮	打9折	包邮	无优惠

☐ 注释：决策表测试适合业务规则复杂的系统（如保险、税务）。

2.2 白盒测试技术 (White-box Testing)

白盒测试需要了解程序内部逻辑和代码结构。

(1) 语句覆盖 (Statement Coverage)

要求每条语句至少执行一次。这是最弱的覆盖标准。

```
# 语句覆盖只要求 a>0 为 True 即可
if a > 0:      #
    print("正") #
else:
    print("非正") #    未覆盖
```

(2) 分支覆盖/判定覆盖 (Branch Coverage)

每个判断条件的 True 和 False 分支都要走到。

```
# 需要 a>0 为 True 和 False 各走一次
if a > 0:  #    True分支    False分支
    ...
else:
    ...
```

(3) 条件覆盖 (Condition Coverage)

每个原子条件的 True/False 都要取到（比分支覆盖更细粒度）。

```
# 需要 a>0 和 b>0 各取 True/False
if a > 0 and b > 0:
    ...
```

(4) 路径覆盖 (Path Coverage)

覆盖程序中所有可能的执行路径（组合爆炸，实际很少用）。

(5) MC/DC（修正条件判定覆盖）

DO-178C（航空标准）要求：每个条件独立影响判定结果。

☐ 注释：实际项目中语句覆盖和分支覆盖最常用。MC/DC 用于航空航天等高安全领域。

第三章 PIE 模型

3.1 PIE 模型概述

PIE 模型解释了故障 (Fault) 如何演变为失效 (Failure)，是理解软件缺陷传播机制的核心理论。

Fault (故障) → Error (错误) → Failure (失效)
依赖 PIE 三条件：

缩写	全称	含义
E	Execution	缺陷代码必须被执行
I	Infection	程序状态必须被感染（产生错误状态）
P	Propagation	错误状态必须传播到输出/界面，被用户感知

☐ 注释：三者缺一不可。如果缺陷代码没有被执行就不会出错，如果错误状态被后续代码修正就不会传播，如果传播不到输出用户就看不到失效。

3.2 经典案例分析

需求：判断成绩 ≥ 60 分为“通过”。

错误代码：

```
// 注意：这是错误的实现！
if (score >= 60) {
    return "通过";
} else if (score >= 60) { // 笔误：应为 score < 60
    return "不通过";
}
```

输入	执行(E)	感染(I)	传播(P)	结果	失效?
75				"通过"	无
55				"通过"	有
80				"通过"	无

score=75：第一个 if 为 True，直接返回“通过”，第二个错误条件未执行（E 不满足）→ 无失效

score=55：第一个 if 为 False，进入 else if，条件又为 False（55 < 60），没有 else 兜底，返回 null 或未定义 → 失效

☐ 注释：PIE 模型解释了为什么有些 Bug 在某些输入下才会暴露——三个条件必须同时满足。

第四章 代码覆盖

4.1 覆盖层次对比

覆盖标准	强度	说明
语句覆盖	★☆☆☆☆	每条语句至少执行一次，最弱
分支覆盖	★★☆☆☆	每个判定的 True/False 都走到
条件覆盖	★★★☆☆	每个原子条件的 True/False 都取到
条件判定覆盖	★★★★☆	条件覆盖 + 分支覆盖
MC/DC	★★★★☆	每个条件独立影响判定结果（航空级）
多条件覆盖	★★★★★	所有条件组合， 2^n 组合，实际很难全测
路径覆盖	★★★★★	所有路径，组合爆炸，几乎不可行

4.2 覆盖率工具

语言	工具	备注
Java	JaCoCo, Cobertura	集成 Maven/Gradle 插件
Python	coverage.py, pytest-cov	<code>coverage run -m pytest</code>
JavaScript	Istanbul (nyc)	<code>nyc mocha</code>
C/C++	gcov, lcov, Bullseye	配合 GCC/Clang 使用
.NET	dotCover, NCover	JetBrains / 商业工具

4.3 覆盖率报告的阅读

Name	Stmts	Miss	Cover	Missing
my_module.py	50	3	94%	12, 38-39
other.py	30	0	100%	
TOTAL	80	3	96%	

☐ 注释：覆盖率目标是指导而非考核。100% 覆盖率 $\neq$ 没有 Bug。行业经验：85%-95% 的分支覆盖率是合理的质量目标。

第五章 GTest 中的常见断言

5.1 ASSERT vs EXPECT

宏	失败后行为	使用场景
<code>ASSERT_*</code>	立即终止当前测试用例	后续测试依赖该条件（如对象非空才能调用方法）
<code>EXPECT_*</code>	记录失败但继续执行	报告更多失败的断言，提高调试效率

□ 注释：优先使用 `EXPECT_*`（可以一次看到所有失败），仅在前置条件不满足会导致程序崩溃时才用 `ASSERT_*`。

5.2 常用断言分类

(1) 布尔断言

```
EXPECT_TRUE(condition);    // 预期条件为真
ASSERT_FALSE(condition);   // 预期条件为假
```

(2) 比较断言

```
EXPECT_EQ(expected, actual); // 相等: expected == actual
EXPECT_NE(val1, val2);       // 不等: val1 != val2
EXPECT_LT(val1, val2);       // 小于: val1 < val2
EXPECT_LE(val1, val2);       // 小于等于: val1 <= val2
EXPECT_GT(val1, val2);       // 大于: val1 > val2
EXPECT_GE(val1, val2);       // 大于等于: val1 >= val2
```

(3) 字符串断言

```
EXPECT_STREQ(str1, str2);    // C风格字符串相等
EXPECT_STRNE(str1, str2);    // C风格字符串不等
EXPECT_STRCASEEQ(str1, str2); // 忽略大小写
```

(4) 浮点数断言

```
EXPECT_FLOAT_EQ(a, b);       // 浮点数近似相等（4个ULP）
EXPECT_DOUBLE_EQ(a, b);      // 双精度近似相等
```

```
EXPECT_NEAR(a, b, 0.001);      // 自定义误差范围
```

□ 注释：浮点数永远不要用 `EXPECT_EQ` 比较！浮点精度误差会导致误判。

(5) 异常断言

```
EXPECT_THROW(statement, exception_type);    // 预期抛出指定异常
EXPECT_NO_THROW(statement);                 // 预期不抛出任何异常
EXPECT_ANY_THROW(statement);                // 预期抛出任意异常
```

(6) 死亡测试 (Death Test)

```
EXPECT_DEATH(func(), "error message"); // 预期以错误信息终止
EXPECT_EXIT(func(), ::testing::ExitedWithCode(1), ""); // 预期退出码为1
```

5.3 完整测试示例

```
#include <gtest/gtest.h>
#include <cmath>

// 被测试函数：计算平方根
double safeSqrt(double x) {
    if (x < 0) throw std::invalid_argument("负数不能开平方");
    return std::sqrt(x);
}

// 测试夹具
class MathTest : public ::testing::Test {
protected:
    void SetUp() override {
        // 每个测试前的初始化
    }
    void TearDown() override {
        // 每个测试后的清理
    }
};

// 测试用例1：正常值计算
TEST_F(MathTest, PositiveNumbers) {
    EXPECT_NEAR(safeSqrt(4.0), 2.0, 0.0001); // 4的平方根应为2
    EXPECT_NEAR(safeSqrt(2.0), 1.4142, 0.0001); // 允许0.0001误差
    EXPECT_DOUBLE_EQ(safeSqrt(0.0), 0.0); // 特例：0
}

// 测试用例2：异常处理
TEST_F(MathTest, NegativeThrows) {
    EXPECT_THROW(safeSqrt(-1.0), std::invalid_argument); // 负数应抛出异常
}
```



```

// 测试用例3: 边界值
TEST_F(MathTest, BoundaryValues) {
    EXPECT_DOUBLE_EQ(safeSqrt(0.0), 0.0); // 最小值
    EXPECT_GT(safeSqrt(0.0001), 0.0); // 极小正数
}

// 测试用例4: 断言对比
TEST_F(MathTest, AssertVsExpect) {
    EXPECT_NEAR(safeSqrt(9.0), 3.0, 0.0001); // 失败继续执行
    EXPECT_NEAR(safeSqrt(16.0), 4.0, 0.0001); // 这条也会执行
    ASSERT_NEAR(safeSqrt(25.0), 5.0, 0.0001); // 失败后终止
    EXPECT_NEAR(safeSqrt(36.0), 6.0, 0.0001); // ASSERT失败后不执行
}

int main(int argc, char** argv) {
    ::testing::InitGoogleTest(&argc, argv); // 初始化GTest框架
    return RUN_ALL_TESTS(); // 运行所有测试
}

```

5.4 常用命令行参数

```

./test_executable --gtest_filter=MathTest.* # 只运行MathTest中的测试
./test_executable --gtest_repeat=10 # 重复运行10次（检查偶发性Bug）
./test_executable --gtest_shuffle # 随机顺序运行
./test_executable --gtest_output=xml:report.xml # 输出XML格式报告

```

□ 注释: `--gtest_filter`

是最实用的参数，开发时可以用来只运行当前关注的测试，避免等待全量测试。

附录：关键概念速查表

概念	一句话解释
软件测试	验证是否按规格构建 + 确认是否满足需求
黑盒测试	不看代码结构，仅关注输入输出
白盒测试	基于代码内部结构的测试
PIE模型	缺陷→故障需要 Execution + Infection + Propagation
等价类划分	输入分组，每组只测代表值
边界值分析	重点测试边界附近的值
语句覆盖	每条语句至少执行一次

分支覆盖	每个判定的 True/False 都走到
MC/DC	航标标准：每个条件独立影响判定
ASSERT_*	失败即终止当前用例
EXPECT_*	失败记录但继续执行

☐ 总结：软件测试是软件质量保证的核心手段。理解基本概念、掌握测试技术、熟悉工具使用是每个软件工程师的必修课。

本文档由原始英文/中文课件整理并翻译，所有 标注处为编者添加的注释和解释。